

Datové typy a proměnné

- **Datový typ** = typ dat, který se dá uložit do nějaké proměnné
- **Proměnná** = pojmenované místo v paměti počítače, které slouží k uchování hodnoty

Základní dělení datových typů

1. Hodnotové typy

- Proměnná obsahuje přímo konkrétní hodnotu
- Při přiřazení do jiné proměnné se vytváří kopie této hodnoty
- **Integer** (celé číslo, např. **10**, **-150**)
- **Float, Double** (číslo s plovoucí desetinnou čárkou, např. **19.99**)
- **Boolean** (logická hodnota **true** nebo **false**)
- **Char** (jeden znak, např. **'A'**)
- **String** (sekvence znaků, např. **"Text"**)

```
let a = 100;  
let b = a; // 'b' nyní obsahuje kopii hodnoty 100  
b = 200; // 'a' stále obsahuje 100, 'b' obsahuje 200. Jsou nezávislé.
```

2. Referenční typy

- Proměnná neobsahuje data, ale referenci (odkaz) na místo v paměti, kde jsou data uložena
- Při přiřazení se kopíruje pouze tato reference
- **Object** (kolekce souvisejících dat ve formátu **klíč: hodnota**)
- **Array** (uspořádaný seznam hodnot)
- **Function** (pojmenovaný blok kódu pro opakované použití)

```
let uzivatel1 = {vek: 30};  
let uzivatel2 = uzivatel1; // 'uzivatel2' odkazuje na stejný  
objekt jako 'uzivatel1'  
uzivatel2.vek = 31; // 'uzivatel1.vek' je nyní také 31, protože obě  
proměnné ukazují na tentýž objekt.
```

Deklarace a inicializace

1. Deklarace

- Proces vytvoření proměnné a zavedení jejího názvu do daného oboru platnosti.
- Ve staticky typovaných jazycích se zde definuje i její datový typ

```
let jmeno; // Deklarace proměnné 'jmeno'
```

2. Inicializace

- První přiřazení hodnoty do deklarované proměnné

```
jmeno = "Tomáš"; // Inicializace proměnné 'jmeno'
```

3. Deklarace s inicializací

- Při deklaraci proměnné přiřadíme rovnou i hodnotu

```
let jmeno = "Tomáš"; // Oba kroky v jednom
```

Statické a Dynamické typování

1. Statické typování (C#, C++, Java)

- Datový typ proměnné je pevně určen při deklaraci
- Nelze jej za běhu programu změnit
- Kontrola typů probíhá během kompilace
- **Příklad (C#):**

```
string jmeno = "Petr";  
  
jmeno = 123; // Chyba kompilace: nejde převést typ  
int na string
```

2. Dynamické typování (JavaScript, Python, PHP)

- Datový typ proměnné je svázán s hodnotou, kterou zrovna obsahuje
- Může se za běhu měnit
- Kontrola typů probíhá za běhu (runtime)
- **Příklad (JavaScript):**

```
let data = "Petr"; // 'data' je typu string  
  
data = 123; // V pořádku, 'data' je nyní typu number
```

Obor platnosti (Scope)

- Definuje oblast kódu, ve které je daná proměnná viditelná a dostupná

```
let globalniPromenna = "Jsem všude";

function mojeFunkce() {
    let lokalniPromenna = "Jsem jen tady";
    console.log(globalniPromenna) //OK
}

console.log(lokálníPromenna) // Chyba: 'lokálníPromenna' is
not defined (není definovaná)
```

1. Globální obor platnosti

- Proměnná je dostupná z jakéhokoliv místa v programu

2. Lokální obor platnosti

- Proměnná je dostupná pouze uvnitř funkce nebo bloku kódu

Imutabilita (Neměnnost) a Konstanta

1. Konstanta

- Zabraňuje změně přiřazení proměnné
- Identifikátor je trvale svázan s původní hodnotou (hodnotové typy) nebo s původní referencí (referenční typy)

```
const cislo = 10;

cislo = 20; // Chyba: Pokus o přiřazení k proměnné typu konstanta

const osoba = {jmeno: "Michal" };

osoba = { jmeno: "Tomáš" }; // Chyba: Nelze změnit referenci
```

2. Imutabilita (Neměnnost)

- Vlastnost hodnoty, kterou po jejím vytvoření nelze vnitřně modifikovat
- Pokud je potřeba změna, musí být vytvořena nová hodnota

Přetypování

- Proces změny hodnoty z jednoho datového typu na jiný
- Rozlišujeme 3 základní druhy

1. Implicitní přetypování

- Jazyk provádí převod automaticky, protože je zaručeno, že nedojde ke ztrátě dat
 - Kdy: Při převodu z datového typu s menším rozsahem na typ s větším
 - Riziko: Žádné. Je to vždy bezpečná operace
 - **Příklad (C#):** Převod **int** na **double**

```
int a = 15;  
double b = a; // Proveďte se automaticky, 'b' má hodnotu 15.0
```

2. Explicitní přetypování

- Programátor musí převod výslovně přikázat, protože při něm může dojít ke ztrátě dat
 - Kdy = Při převodu datového typu s větším rozsahem na typ s menším rozsahem
 - Riziko: Ano, hrozí ztráta dat nebo přesnosti. Programátor přebírá zodpovědnost
 - **Příklad (C#):** Převod z **double** na **int**

```
double a = 15.9;  
int b = (int)a; // Nutné explicitně uvést (int). 'b' má hodnotu 15
```

3. Konverze pomocí metod

- Používá se pro převod mezi nekompatibilními typy, jako je text (**string**) a číslo (**int**)
- Vyžaduje zavolání specializované funkce
 - Kdy: Když typy nemají přímou číselnou souvislost
 - Riziko: Ano, operace může selhat a vyvolat chybu (výjimku), pokud vstupní data nejsou ve správném formátu
 - **Příklad (C#):** Převod **string** na **int**

```
string textCisla = "42";  
int cislo = int.Parse(textCisla); // Volá se metoda Parse(),  
'cislo' má hodnotu 42  
string spatnyText = "ahoj";  
int jineCislo = int.Parse(spatnyText); // Toto by vyvolalo  
chybu, protože string "ahoj" není možné převést na int
```